

東京 2025
LINUX PLUMBERS CONFERENCE

TOKYO, JAPAN / DECEMBER 11-13, 2025

Push-Based Load Balancing
For Fair Tasks

K Prateek Nayak <kprateek.nayak@amd.com>



東京 2025
LINUX PLUMBERS CONFERENCE
TOKYO, JAPAN / DEC. 11-13, 2025

Quick Overview: What do we want to achieve?

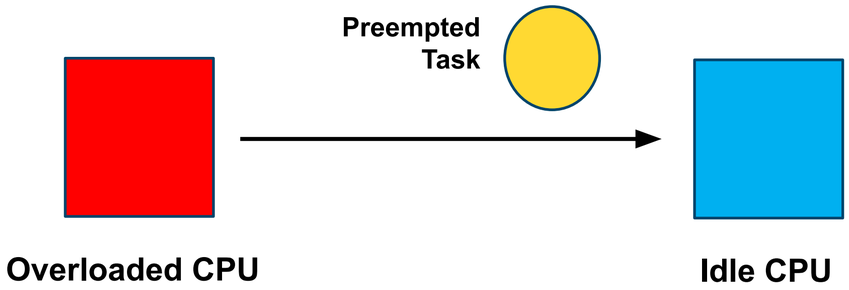
In a flat hierarchy with many groups, periodic balance can take a long time to kick in.

- **Load balancing period scales with the number of CPUs.**
- At any point in time, **only a single CPU in the group can go past should_we_balance()** (except in the case of newidle balancing).

To reduce wait time in presence of idle CPUs, an overloaded CPU can proactively push a preempted task to an idle CPU.

Requirements

- **Cheap indicator** to signal the presence of an idle CPU
- A **fast way to lookup an idle CPU**
- A **bailout mechanism** when CPU is overutilized / when pushing attempts keep failing.



Preparation: Optimizing sched_domain_shared Assignments

Problem statement

- **Per-CPU sched_domain_shared objects are allocated for every sched_domain during __sdt_alloc()**
- sd->shared is only assigned for SD_SHARE_LLC domains and **only sd_llc_shared is ever used**.

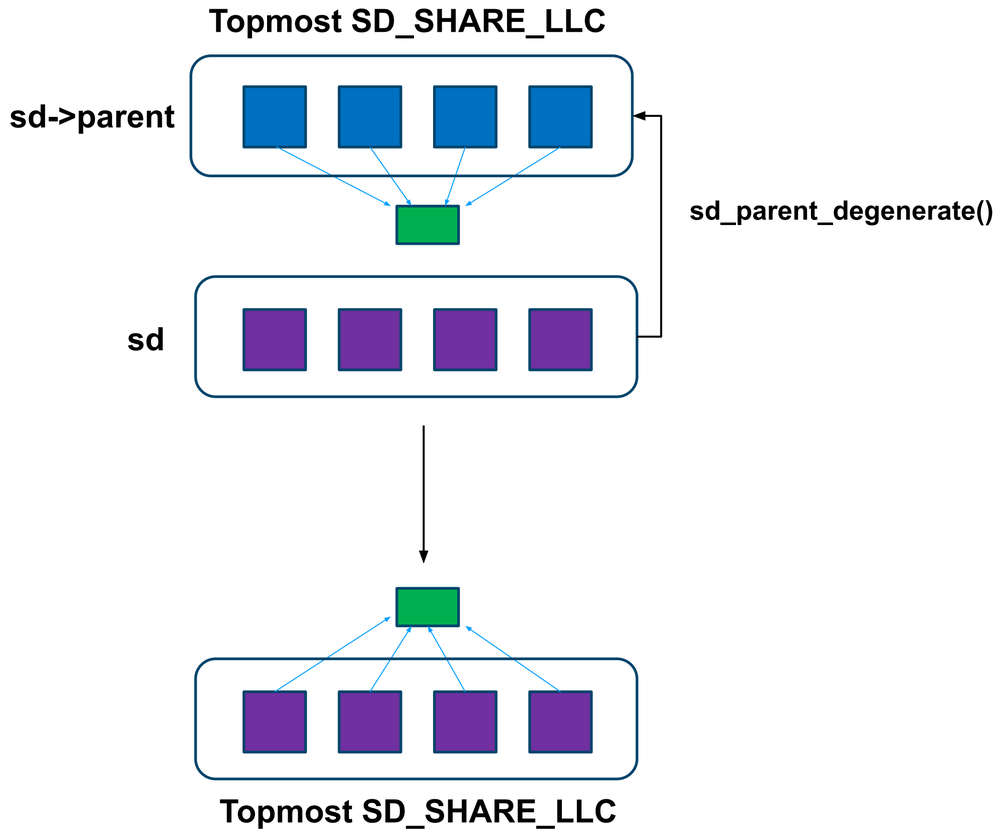
Does it matter?

- commit [f79c9aa446d6](f79c9aa446d6) **("x86/smpboot: avoid SMT domain attach/destroy if SMT is not enabled")** optimized for unnecessarily allocating and destroying a known redundant domain.
- **sd->shared for SMT and CLUSTER domains are assigned but never used.**
- **Embedding a cpumask** (or two) in the sched_domain_shared object **can blot its size** – more wasted space, more memory pressure during domain allocation.

Can we do better?

Potential solution

- **Allocate a single set of per-CPU of sched_domain_shared** if an SD_SHARE_LLC domain is detected.
- **Assign them to the topmost SD_SHARE_LLC domain** before degeneration.
- **Pass down the sd->shared to child if sd is degenerated.**



Preparation: Breaking down nohz.idle_cpus_mask

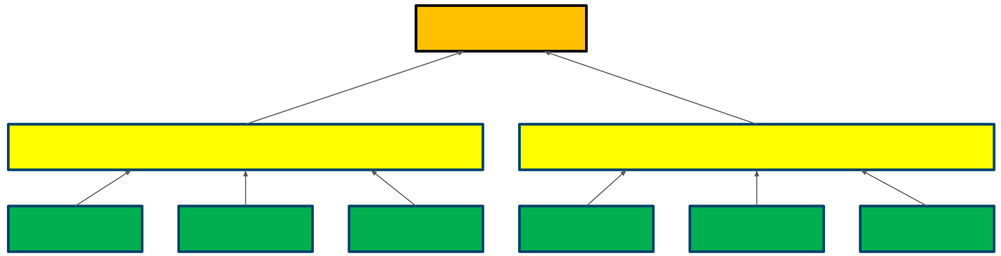
Problem statement

- “nohz.idle_cpus_mask” is a global mask used to iterate over the nohz idle CPUs in the system. **(Fast way to look up an idle CPU)**
- “nohz.nr_cpus” is a global count of nohz idle CPUs modified atomically.
- **In a large system with multiple CPUs transitioning in and out of nohz idle, this mask can ping-pong between CPUs of different cache domains / NUMA domains.**

Peter suggested **breaking down the global “nohz.idle_cpus_mask” and instead maintain it per LLC domain** (https://lore.kernel.org/lkml/20250410102945.GD30687@noisy.programming.kicks-ass.net/)



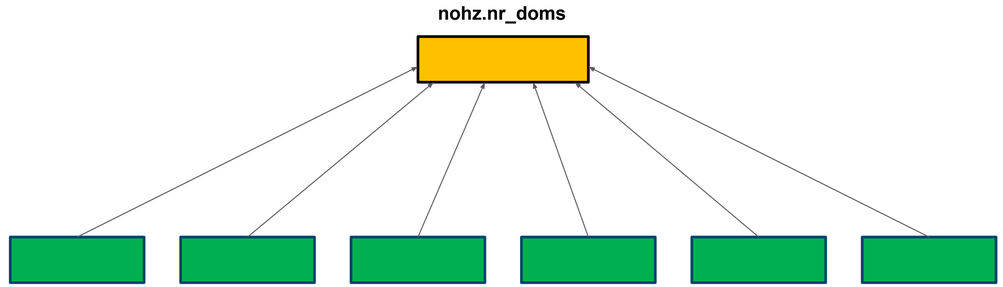
Breaking down nohz.idle_cpus_mask: Two Approach



Propagate signal upwards

Atomic operations scale with domain depth and number of intermediate groups

Implemented in the RFC



Update global indicators at boundary conditions

Atomic operations proportional to number of leaf nodes

In both cases, atomic updates at higher level are only done at boundary conditions of (0 -> 1) and (1->0) transitions at lower levels



Global list that chains all the sd->shared keeping track of idle CPUs

Push-Based Load Balancing: The Learnings

- Relying on per-LLC **nohz_idle_cpus_mask** alone is not sufficient for a variety of workloads – there can be multiple CPUs in shallow idle state which can be an equally good push target.
  Cons: The cost of traversing the entire sched_domain_span() can be expensive on systems with larger LLC.

- **double_rq_lock** can be expensive if multiple CPUs decide to converge on the same idle CPU.
  Question: Can __ttwu_queue_wakelist() be extended for load balancing where attach_task() for idle balance can be offloaded to the idle target CPUs?

- Pushing tasks can keep failing if the task is affined to a small subset of CPUs or if the dynamic nature of the system makes the idle indicator unreliable.
  Bailout mechanism can help prevent these pathological cases of continuous failure after and attempt.
  Current mechanism piggybacks on the **SIS_UTIL's "nr_idle_scan"** to limit the number of push attempts.

Food for thought: If an eligible delayed task is found during load balancing, would it be beneficial to block_task() instead of migrating it?

Benchmark	Only nohz_idle_cpus	Search entire sched_domain_span()
schbench-new-64 threads (Request latency)	-4.79%	7.82%
Hackbench 16-groups (Runtime)	-1.82%	-8.57%

Benchmark	No Rate Limiting	Rate Limiting
schbench-new-32 threads (Request latency)	-14.88%	0%
schbench-new-128 threads (Request latency)	-9.75%	-1.5%
schbench-old-256-tthreads (99%ile latency)	-7.45%	15.98%

Results from runs on a 3rd Generation EPYC system (64C/128T) on a v6.18-rc7 based kernel

Push-Based Load Balancing: Limited Improvements (so far)

DeathStarBench is the only clear winner out of all the benchmarks run with the push-based load balancing.

Few failed experiments:

- **Shallow Idle CPUs Mask**: Keep a track of all CPUs switching to the idle thread and use it for push and for task wakeups.
  - **Advantages**:
    - Allows for **easy tracking of idle CPUs**, like sched_ext's idle tracking mechanism.
    - Can be optimized like nohz.idle_cpus_mask to only clear CPU on tick and not at idle exit.
  - **Disadvantages**
    - The **cost of maintaining the cpumask, even for 16CPUs, is very visible in benchmarks like tbench** (**~15% regression** at fully loaded)

- **Overloaded CPUs Mask**
  - Like the **Shallow Idle CPUs Mask** except the CPU is set on the mask when rq->nr_running transitions from <= 1 to > 1 (**same as the rd->overloaded indicator**) and cleared when idle task is picked.
  - Unsurprisingly, like **Shallow Idle CPUs Mask**, this too was found **too expensive to maintain** even with no users**.**

DeathStarBench	%improvements
1x Scaling – Pinned to 1 CCX	24%
2x Scaling – Pinned to 2 CCX	12%
3x Scaling – Pinned to 4 CCX	17%
6x Scaling – Pinned to 8CCX	4%

Results from runs on a 3rd Generation EPYC system (64C/128T) on a v6.18-rc7 based kernel

Newidle Balancing: Faster Intra-NUMA Balancing

On a **CONFIG_PREEMPTION** kernel, newidle balancing bails out after pulling a single task.

```
sched_balance_newidle()
  sched_balance_update_blocked_averages();
  for_each_domain(sd)
    sched_balance_rq(rq, sd)
      sched_balance_find_src_group();
      sched_balance_find_src_rq();
      detach_tasks()
        …
        detached++;
        if (idle == CPU_NEWLY_IDLE)
          return detached;
```




```
for_each_sd(sd)
  for_each_cpu_andnot(cpu, span(sd), span(sd->child))
    if (cpu_overloaded(cpu))
      detach_one_task()
```

Question: Can we instead just simply traverse the CPUs of sched_domain_span() searching for an overloaded CPUs to pull tasks from?

Newidle Balancing: Interesting Discoveries

- When traversing CPUs directly, it is easy to bail out of newidle balancing when a task is woken up or a pending wakeup is detected on balancing CPU:

```
for_each_cpu_andnot(cpu, span(sd), span(sd->child))
  if (this_rq->nr_running || this_rq->ttwu_pending)
    break;
```

However, this is found to cause up to 10% regression across various workloads.

- The schedstats point to changing wakeup behaviors where the tasks seem to wakeup on local CPU more often.
- **The chicken and egg problem**

```
idle_cpu(cpu)
  rq = cpu_rq(cpu)
  if (rq->curr != rq->idle)
    return false;
  …
```

```
pulled_task = sched_balance_newidle();
if (!pulled_task)
  return NULL;
put_prev_set_next_task(prev, rq->idle
rq->curr = rq->idle;
```

Task wakeups don’t explore newidle balancing CPUs, and newidle balancing CPU won’t bail out of the search without a task wakeup.

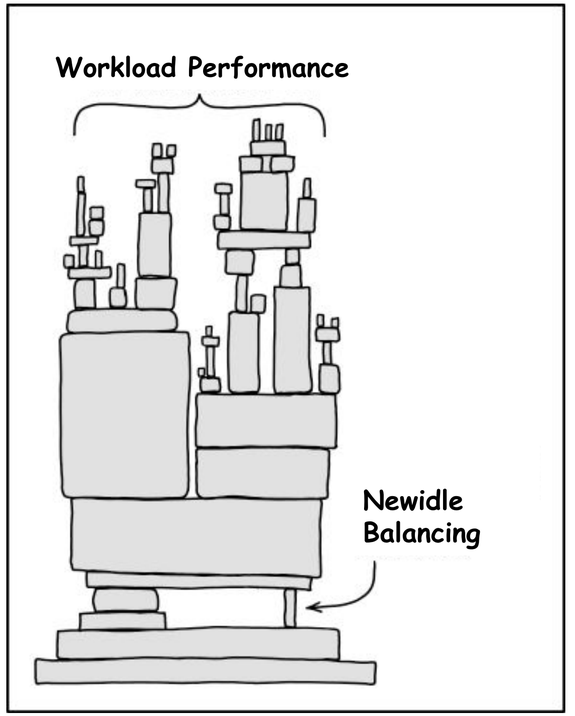


Image adapted from xkcd’s Dependency (https://xkcd.com/2347/)

Newidle Balancing: Implementation and Results

- Only check if CPU turned busy before CPU traversal

```
if (this_rq->nr_running || this_rq->ttwu_pending)
    break;
for_each_cpu_andnot(cpu, span(sd), span(sd->child))
    …
```

Keep the bailout based on **max_newidle_lb_cost** and **NI_RANDOM** intact.

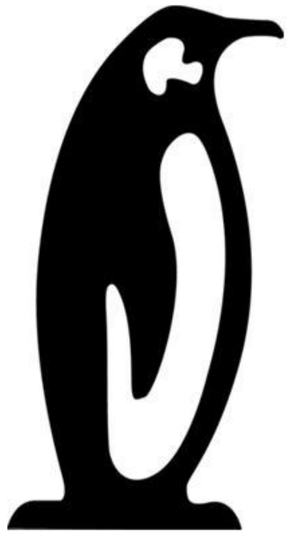
- The optimization leads to a **mixed bag of results**:
  - sched-messaging regresses when system is nearly fully loaded.
  - schebench request latency spikes when system is fully loaded but Requests per Second (RPS) stays the same.
  - Old schbench's tail latency spikes up as system starts ramping towards fully busy.
  - YCSB-mongodb and YCSB-Cassandra both regress by a couple %.
  - DeathStarBench shows major improvements.

DeathStarBench	%improvements
sched-messaging 16-groups	-14.78%
Old-schbench 64-messenger tail latency	-8.59%
New schbench 128-messenger request latency	-16.14%
1x Scaling – Pinned to 1 CCX	24%
2x Scaling – Pinned to 2 CCX	12%
3x Scaling – Pinned to 4 CCX	17%
6x Scaling – Pinned to 8CCX	4%

Results from runs on a 3rd Generation EPYC system (64C/128T) on a v6.18-rc7 based kernel

Next steps

- **Any takers for the nohz.idle_cpus_mask optimization?**
- Profile the workloads to find better limits for attempting push-based load balancing.
- Analyze the trade-off of removing “rq->curr == rq->idle” requirement in idle_cpu(). **Is there a better alternative?**
- What exactly does newidle balance want?
  Can we replace RANDOM_NI with something more principled?
- More testing and analysis on a single LLC per socket systems.
- Look more into busy balancing and possible optimizations.



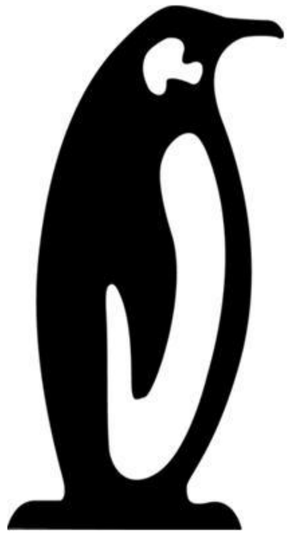
東京 2025

ありがと (Thank You)

To everyone hacking, reviewing, testing, and maintaining the Linux kernel

Special thanks to the Maintainers, Reviewers, and Contributors of the Linux scheduler, and everyone here for putting up with my crazy ideas (^_^)





東京 2025
LINUX PLUMBERS CONFERENCE

TOKYO, JAPAN / DECEMBER 11-13, 2025

